

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 3 — Engineering Foundations

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Today's focus and learning goals



Repository workflow and safe change management



Branches, commits, and issue tracking



Reading the starter codebase as one connected workflow



Unit, integration, and smoke tests — evidence, not ritual



Practical refresher: HTTP, JSON, SQL



Individual Readiness Lab expectations

Why Engineering Foundations Matter

Most avoidable project failures start small

- Editing the wrong repository or branch
- Changing multiple things at once
- Not knowing whether the system worked before the change
- Testing only the happy path manually
- Forgetting to push work
- No record of why a change was made
- Setup that works only on one laptop

The goal is not caution for its own sake.

- The goal is to make change:
- understandable
- reversible
- testable
- reviewable



The Repository Is a Shared Engineering Record

A repository is more than a folder of code

It should record:

1. Current system behavior
2. Source code and configuration
3. Tests and verification logic
4. Documentation and setup instructions
5. Change history
6. Decisions and assumptions
7. Known limitations

Ask:

- If another teammate opened this repository tomorrow, could they understand how to run and inspect the system?



Git Mental Model: Working Tree, Commit, Remote

Basic change flow

edit files

↓

git status (*show current state of the working tree*)

↓

git add (*place selected changes into the staging area*)

↓

git commit (*saves a snapshot in your local Git history*)

↓

git push (*sends those commits to the shared remote repository*)



Three places to keep distinct

Place	Meaning
Working tree	Files currently on your laptop
Local Git repository	Your local history of commits
Remote repository	Shared repository on GitHub

The Safe Change Loop

Before, during, and after every bounded change

1. Start clean
2. Understand the existing behavior
3. Make one focused change
4. Inspect the diff
5. Run relevant verification
6. Commit with a useful message
7. Push and confirm

Useful commands

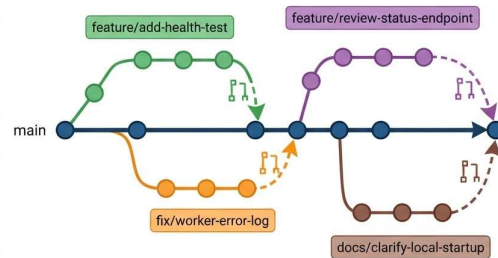
```
git status
```

```
git diff
```

```
git log --oneline -5
```

```
git branch --show-current
```





Branches: A Safe Place to Change Things

A branch is an isolated line of work

Use a branch when you want to:

- Add a feature
- Fix a bug
- Experiment safely
- Keep work separate from stable code
- Prepare a change for review

Example

```
git switch -c readiness/add-health-test # create + switch
```

...make one focused change, verify...

```
git push -u origin readiness/add-health-test
```

open a Pull Request on GitHub → review your own diff → merge

```
git switch main && git pull  
# bring the merge to your laptop
```

Good branch names

- readiness/add-health-test
- fix/worker-error-log
- feature/review-status-endpoint
- docs/clarify-local-startup

Commits and Issues Are Communication

1. Add date "Sep 2026" in README.md

2. `git add README.md`

3. `git commit -m "added current date on README"`

4. `git push`

A commit should answer:

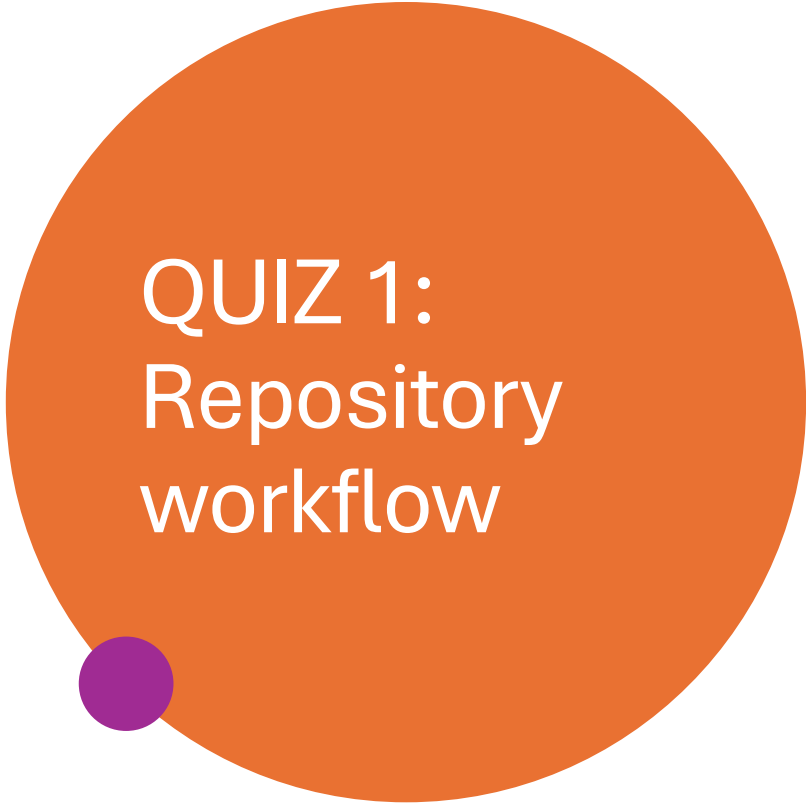
- What changed?
- Why did it change?
- Is the change focused?

Weak commit messages

- update
- fix
- final final
- changes
- week 3

Better commit messages

- Add test for API live-health endpoint
- Persist source field for submitted cases
- Document local reset workflow
- Handle worker failure with structured error log
- **Issue tracking asks:**
- What work exists, who owns it, and what does "done" mean?

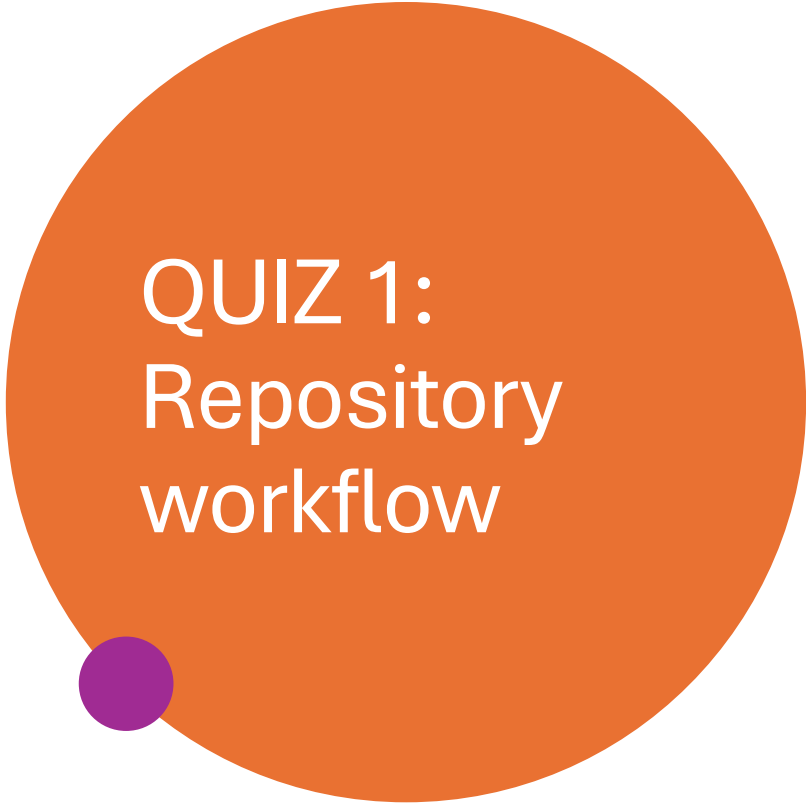


QUIZ 1: Repository workflow



Q1. You edited a file, ran git add and git commit, and see the commit in git log. Your teammate says the change is not on GitHub. What is the most likely reason?

- A) The commit failed silently
- B) You have not run git push
- C) GitHub only shows commits made on main
- D) The file is in .gitignore

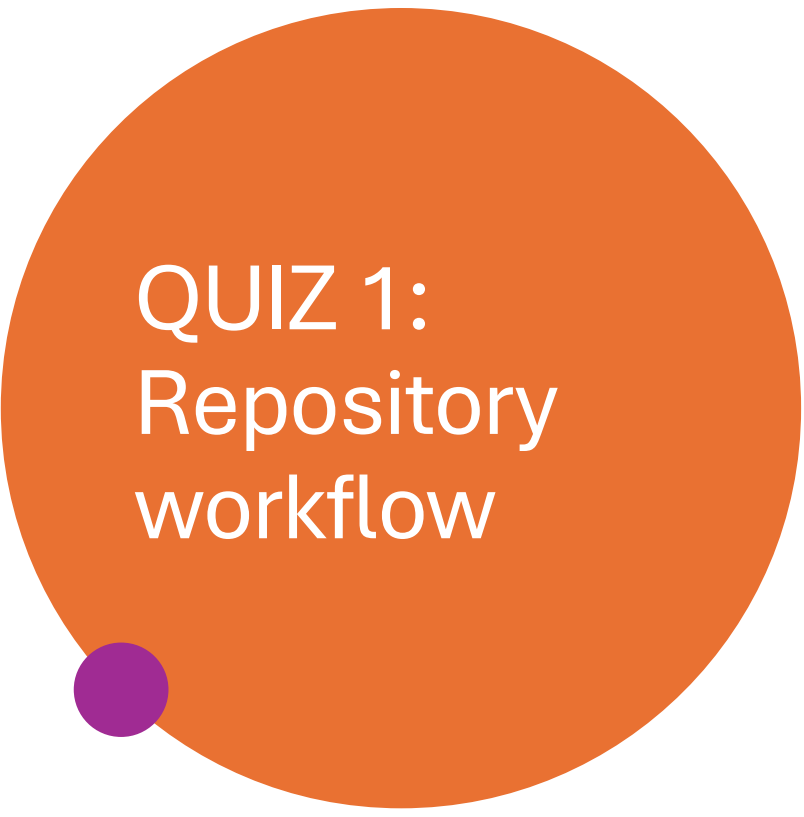


QUIZ 1: Repository workflow



Q2. You want to check that you cloned *your own assigned repository* and not the original course template. Which command answers this most directly?

- A) git status
- B) git log --oneline -5
- C) git remote -v
- D) git branch --show-current

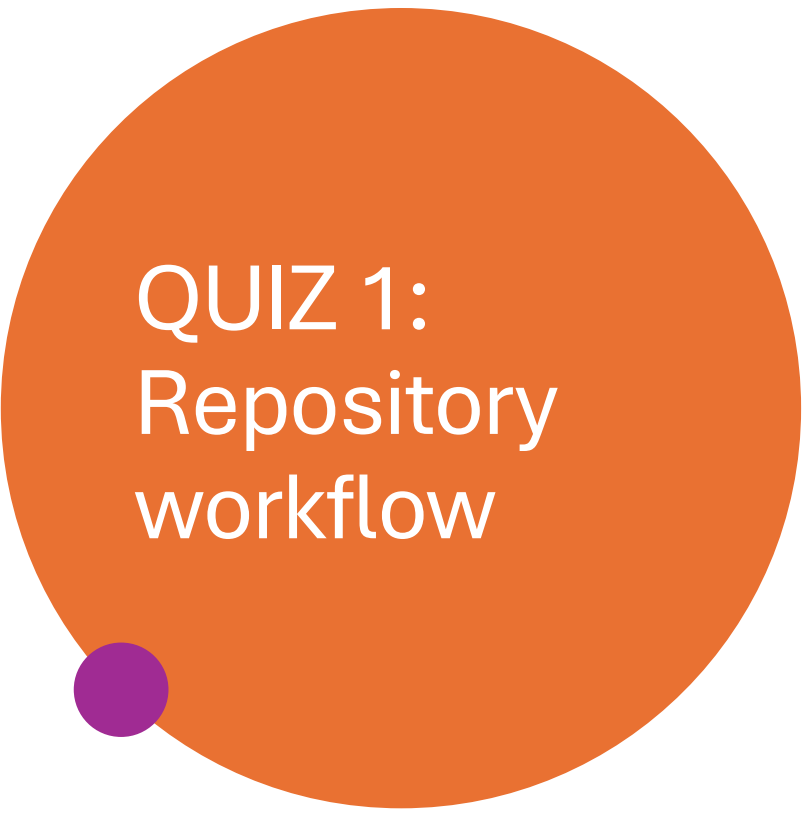


QUIZ 1: Repository workflow



Q3. Which commit message best follows the course guidance?

- A) update
- B) final final
- C) Add test for API live-health endpoint
- D) week 3 stuff



QUIZ 1: Repository workflow



Q4. Why does the course ask you to make your readiness change on a branch rather than directly on main?

- A) Branches make the code run faster
- B) Git does not allow commits on main
- C) A branch isolates the change so main stays stable and the change can be reviewed before merging
- D) Tags can only be created on branches

Read the Starter Repository as a System

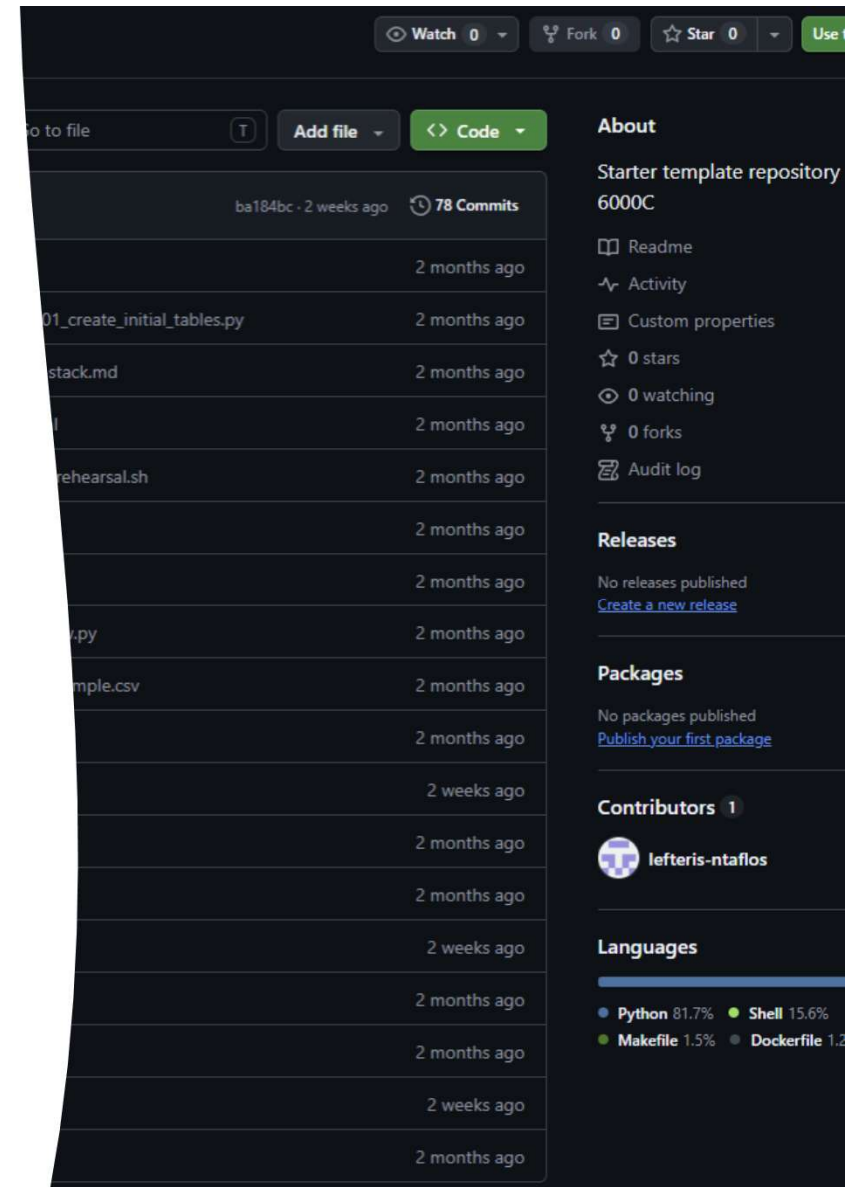
Do not begin by reading every file.

Start by tracing one workflow:

Request → services/api/app/main.py (POST /cases)
→ services/common/models.py (Case row, Job row in PostgreSQL)
→ services/worker/app/main.py (claims job, attempts += 1)
→ services/ai/app/main.py (POST /triage → label, summary, confidence)
→ worker writes ai_label / ai_summary / ai_confidence, status = triaged
→ GET /cases/{id} exposes the result; JSON logs show job_claimed / job_completed

Questions to ask

- Where does the request enter?
- Where is state stored?
- What triggers background work?
- Where is the AI function called?
- How are errors represented?
- Which tests cover the workflow?



Testing is evidence, not ritual: unit, integration, smoke

Testing answers a practical question:

- How do we know this change did not break the behavior we rely on?

Testing helps us check:

- expected behavior
- invalid input handling
- state changes
- regression risk
- integration between components
- startup and basic availability

Tests do not prove perfection.

- They provide credible evidence.

Type	Main question	Example	In the starter
Unit test	Does one small piece behave correctly?	Category-mapping function returns expected label	tests/unit/test_ai_service.py calls triage_text() directly
Integration test	Do connected components work together?	API request creates a database record	tests/integration/test_api_case_flow.py uses FastAPI TestClient + SQLite
Smoke test	Does the running system basically work?	Containers start and health endpoint responds	tests/smoke/test_smoke_cases.py against SMOKE_BASE_URL

What Makes a Test Useful?

A useful test is:

1. Specific about expected behavior
2. Easy to read
3. Deterministic
4. Related to a real requirement
5. Fast enough to run regularly
6. Clear when it fails

Example

```
def test_live_health_returns_ok(client):  
    response = client.get("/health/live")  
  
    assert response.status_code == 200  
    assert response.json()["status"] == "ok"
```

Practical Refresher: HTTP, JSON, and SQL

HTTP

GET Read a resource
POST Create a resource
PATCH Update part of a resource
DELETE Remove a resource

JSON

```
{  
  "title": "Payment issue",  
  "description": "Customer reports a duplicate charge."  
}
```

SQL mindset

- A table stores records.
- A row is one record.
- A primary key identifies it.
- A foreign key connects related records.

Code	Meaning	In the starter
200	OK	GET /cases/{id}, /health/live
201	Created	POST /cases
404	Not found	unknown case or job id
422	Validation error (JSON does not match schema)	title shorter than 3 chars
500	Server error — read the logs	/health/ready when the DB is down

Customers table

customer_id	name	email
101	Alice	alice@example.com

Orders table

order_id	customer_id	product
5001	101	Laptop
5002	101	Keyboard



QUIZ 2: Testing, HTTP, SQL

Q1. A check starts all containers with docker compose up --build, then requests /health/live and expects HTTP 200. Which type of test is this?

- A) Unit test
- B) Integration test
- C) Smoke test
- D) Regression test



QUIZ 2: Testing, HTTP, SQL

Q2. Which of the following is **not** a property of a useful test?

- A) It is specific about the expected behaviour
- B) It sometimes passes and sometimes fails depending on timing
- C) It is related to a real requirement
- D) It is clear about what went wrong when it fails



QUIZ 2: Testing, HTTP, SQL

Q3. The API schema requires title to be at least 3 characters. A client sends POST /cases with "title": "ab". Which status code should the API return?

- A) 200
- B) 201
- C) 404
- D) 422

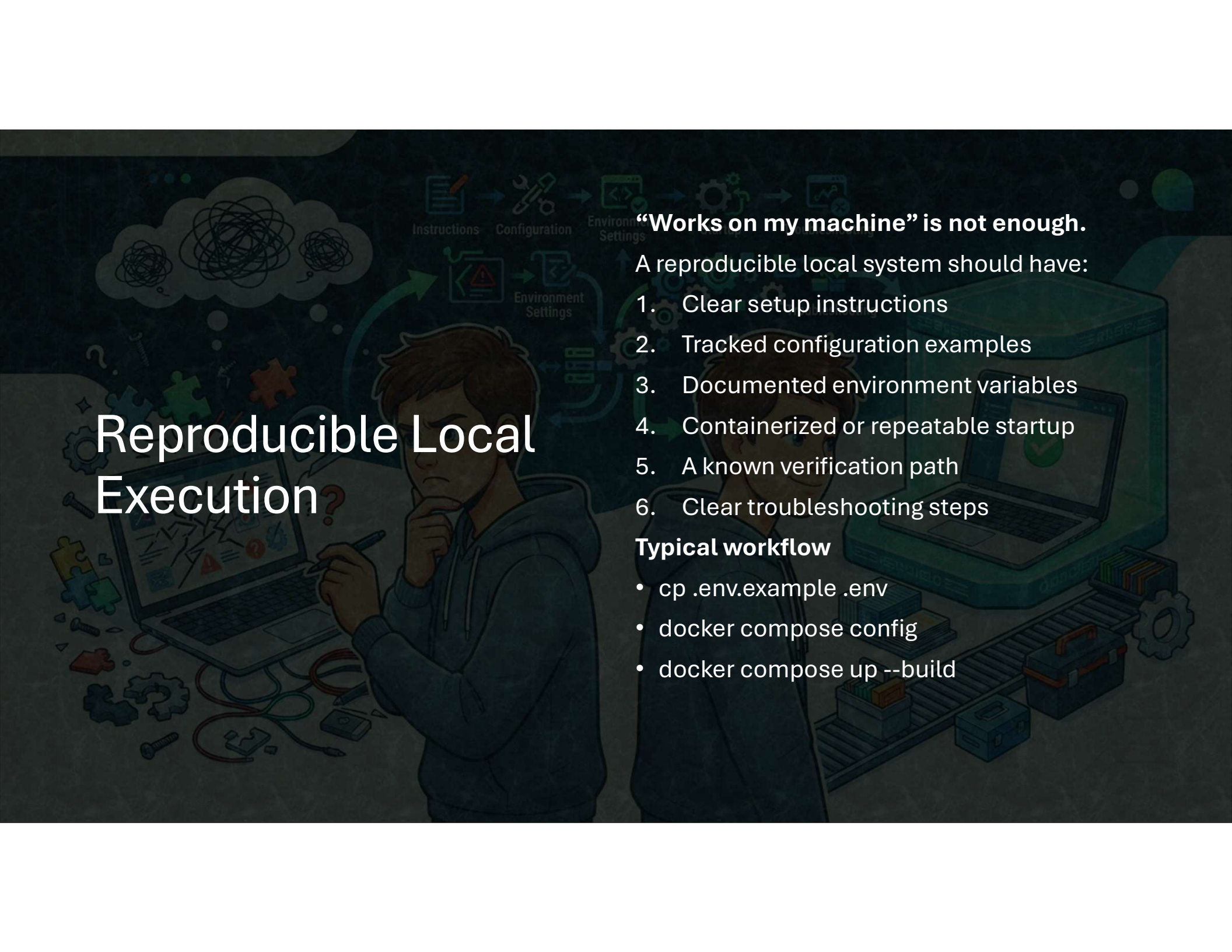


QUIZ 2: Testing, HTTP, SQL

Q4. In the starter, the jobs table has a case_id column that refers to cases.id. In the jobs table, case_id is a:

- A) Primary key
- B) Foreign key
- C) Index only
- D) JSON field

Reproducible Local Execution

A dark-themed illustration featuring a person in a hoodie sitting at a desk, looking thoughtful with a hand on their chin. On the desk is a laptop displaying a complex diagram with a red warning triangle. To the left of the laptop are various mechanical parts like gears, bolts, and puzzle pieces. Above the person, a thought bubble contains tangled lines. In the background, a workflow diagram shows steps: 'Instructions' (notepad icon), 'Configuration' (wrench icon), 'Environment Settings' (code editor icon), and 'Environment Settings' (code editor icon with a red triangle). Arrows connect these steps in a cycle. To the right, another person is partially visible, looking at a laptop that shows a green checkmark. Further right, there's a stack of server racks and a toolbox with a gear.

“Works on my machine” is not enough.

A reproducible local system should have:

1. Clear setup instructions
2. Tracked configuration examples
3. Documented environment variables
4. Containerized or repeatable startup
5. A known verification path
6. Clear troubleshooting steps

Typical workflow

- `cp .env.example .env`
- `docker compose config`
- `docker compose up --build`



Individual Readiness Lab: what it requires

- Individual checkpoint. Demonstrate that you can: run the repo · use Git/GitHub correctly · make one bounded change · verify it · document it.
- Repository must contain: the bounded change · a test or verification update · submissions/week03/README.md (create the folder; copy from templates/submissions/week03-README-template.md) · tag w03-readiness pushed to GitHub.
- Canvas text entry: Repository · Checkpoint tag w03-readiness · Commit SHA · What I changed · How I verified it · AI Use Statement.
- Tag only after the final change is verified, documented and pushed:

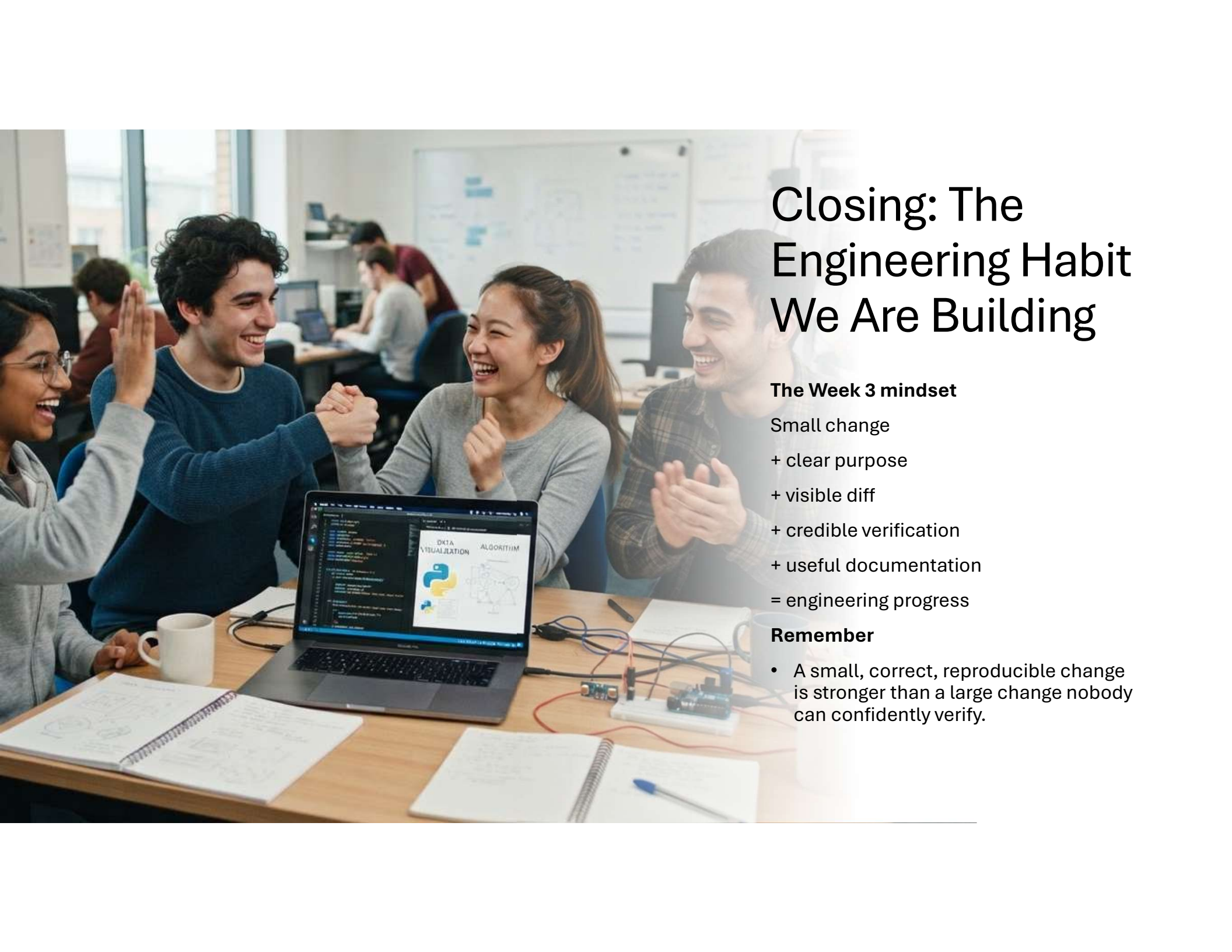
```
git tag -a w03-readiness -m "Week 3 readiness checkpoint"
git push origin w03-readiness
git rev-parse w03-readiness^{commit}    # ← the SHA to
paste into Canvas
```
- This is not a mini-project.

Lab Session vs. Assignment



Week 3 Lab	Individual Readiness Assignment
Guided practice	Individual assessed work
Learn the workflow	Demonstrate the workflow
Inspect, discuss, rehearse	Make and submit your own bounded change
May work alongside classmates	Must represent your own work
Produces preparation notes	Produces repository evidence, tag, and Canvas record

Do not confuse preparation with submission.



Closing: The Engineering Habit We Are Building

The Week 3 mindset

Small change

+ clear purpose

+ visible diff

+ credible verification

+ useful documentation

= engineering progress

Remember

- A small, correct, reproducible change is stronger than a large change nobody can confidently verify.